

Velora MERN Duck Log

Alignment protocol, evidence model and project session record

Version 2.0 | 10 October 2026 | Swayam Jain

01 / Current truth and handoff

Version 2.0 - MERN rebuild progress authority. Read with VELORA_MERN_MASTER and VELORA_MERN_ARCHITECTURE.

Field	Verified status on 10 October 2026
Owner / brand	Swayam Jain / Velora.
Repository	https://github.com/Swayamjain1240/Velora
GitHub content	VERIFIED EMPTY: size 0, no branches or commits at the latest read.
Chosen stack	JavaScript MERN: MongoDB/Mongoose, Express, React/Vite, Node.
Current implementation	NOT STARTED in this GitHub repository.
Current part	Part 1 planned; no completion claim.
External providers	None selected for storage, AI/OCR, queue, notification or hosting.
Data boundary	Synthetic data only; real-patient use not authorized.
Next action	Run Part 1 prompt: Duck, preserve local files, initialize secure MERN foundation.

Previous architecture or code discussed elsewhere is historical context only. It is not implementation evidence for this empty MERN repository. If local or remote content appears later, inspect and classify it rather than overwriting or assuming equivalence.

Authoritative references

- VELORA_MERN_MASTER: product scope, VEL requirements, ten parts, Security-25 and UI invariant.
- VELORA_MERN_ARCHITECTURE: JavaScript MERN system design and technical boundaries.
- VELORA_MERN_DUCK_LOG: evidence, review protocol, decisions, progress and next action.
- VELORA_MERN_10_BUILD_PROMPTS.md: executable part-by-part implementation prompts; it cannot override the three authorities.

02 / Duck protocol - establish evidence

When the owner says Duck, starts a part or asks for alignment, perform this review before implementation.

- Read the current three authoritative documents, README, AGENTS/repository instructions, active part prompt, API contracts and previous part review.
- Inspect actual repository root, branch, remote, HEAD, dirty tree, recent commits, PR/check state and untracked files. Preserve user changes.
- Inspect source routes/controllers/services/policies/models/indexes, client routes/components, config, dependencies, .gitignore, CI and tests. Never print secrets or sensitive payloads.
- Verify Mongo/environment/storage/provider assumptions from code, not old summaries.
- Map actual behavior to VEL IDs, Security-25 and the current part exit gate. A file existing is not proof it runs; a passing test is evidence only for its assertions and revision.
- If files, repository, service or test environment is inaccessible, report BLOCKED or UNVERIFIED. Continue useful document/code review without inventing evidence.

Evidence precedence

Current explicit owner decision -> current authoritative docs -> current repository instructions -> inspected code/config -> tests on the exact revision -> CI logs. Old chats, superseded docs and tests from another revision are context, not current proof.

03 / Duck classification and response

Status	Meaning
VERIFIED	Observed implementation plus appropriate passing evidence on the stated revision.
IMPLEMENTED / UNVERIFIED	Code exists but required execution evidence is missing or stale.
PARTIAL	Only some current acceptance criteria are met.
PLANNED	Correctly deferred work with no implementation claim.
DRIFT	Behavior contradicts requirements, architecture or owner decision.
BLOCKED	Missing access, environment, provider decision or unresolved material policy prevents verification/progress.

Act on findings

On track: state evidence and continue the currently authorized task. Ordinary in-scope drift: make the smallest reversible correction, run affected tests, update docs and continue. Critical isolation, approval, secret, file, data-loss or stale-schedule risk: stop dependent features, correct it and verify. Material clinical/product/provider ambiguity: prepare concrete options and identify the needed owner decision; never invent clinical policy.

Duck does not by itself authorize destructive data deletion, irreversible history rewrite, production deployment, paid service use, external messages or real-patient data changes. Do not remove tests, weaken requirements, force-push or silently start the next part to make status look complete.

Verdict

Use ON TRACK, DRIFT FIXED, ACTION REQUIRED or BLOCKED. Do not say DRIFT FIXED until the correction passes appropriate verification. Passing checks never means zero possible bugs.

04 / Part gate and Security-25 evidence

Start gate

- Previous part is merged/reviewed as required, documentation is current and no unresolved dependent security failure remains.
- Scope and VEL IDs are named; user flow -> entities -> permissions -> APIs -> services/transactions -> UI -> failures -> evidence is written.
- Any new external API disclosure is complete before SDK installation or network use.

During work

- After coherent work or about 300 meaningful LOC, test and inspect diff before one meaningful commit/push. Do not optimize for commit count.
- Test denials and failures with each feature; do not wait for a final security patch.
- Keep .env/private files ignored and documentation/API contracts synchronized.

Completion gate

- Feature, failure, concurrency/security, Mongo integration, client build and browser viewport checks pass on the final executable revision.
- Dependency/runtime audit and Git tree/history secret scans are recorded. Findings are fixed or explicitly classified with impact.
- All 25 controls are marked PASSED, FAILED, BLOCKED or NOT APPLICABLE with evidence; no blank assumptions.
- README, three authorities, API contract, part review and session entry match actual code. Next part remains paused until authorized.

05 / Duck report and session templates

```
DUCK REVIEW - date/time/timezone
Repository / branch / commit / dirty state:
Current part and VEL IDs:
Verdict: ON TRACK | DRIFT FIXED | ACTION REQUIRED | BLOCKED
Evidence inspected:
Requirement findings: ID | expected | actual | status | evidence
Security-25 findings: # | status | evidence | remaining risk
Corrections made:
Verification: command/run | result | revision
Failed, blocked or not-run checks:
External API disclosures:
Documentation updated:
Unresolved decisions/risks:
Next authorized action:
```

Session entry

```
Date/time and timezone:
User request and authorization:
Repository/branch/commit/working tree:
Part, milestone and VEL IDs:
Files/evidence inspected:
Changes and meaningful commits:
Tests and exact observed outcomes:
Security/dependency/secret findings:
Duck verdict:
Open decisions/blockers:
Next concrete action:
```

Append entries. Do not rewrite a prior failure as if it never occurred; add the later fix and evidence. Exact commands and CI run links belong in the part review/session entry, but secrets and patient/model payloads do not.

06 / Initial MERN rebuild entry

10 October 2026 - stack reset and durable prompts

Owner selected JavaScript MERN and dropped the earlier implementation approach for the new build. Generated ten standalone part prompts covering Parts 1-10. Every prompt starts with Duck, repeats Security-25, protects MONGODB_URI/.env, requires .gitignore, uses meaningful commits and enforces the one-viewport application rule.

Read-only GitHub inspection of <https://github.com/Swayamjain1240/Velora> returned repository size 0, default branch name main, no branch objects and no commit history. Therefore no MERN application feature is currently VERIFIED. Part 1 is PLANNED. Any local files discovered at implementation time must be preserved and reclassified.

Created this three-document authority set for future Duck reviews: Master, Architecture and Duck Log. These documents do not initialize Git, install dependencies, create MongoDB credentials, call an external provider, deploy or authorize real-patient data.

Current Duck verdict

ON TRACK FOR PLANNING. Scope, stack, security rules, ten-part sequence and evidence method are defined. IMPLEMENTATION NOT STARTED. Next action requires the owner to issue the Part 1 prompt.

Standing secret rule

MONGODB_URI and every real credential belong in an untracked server .env or managed runtime secret. Only .env.example with names and placeholders may be committed. Before the first push, inspect tracked files and scan Git history/tree for leaks.